EXECUTIVE SUMMARY · REFERENCE PATTERN FOR SAP BDC CUSTOMERS

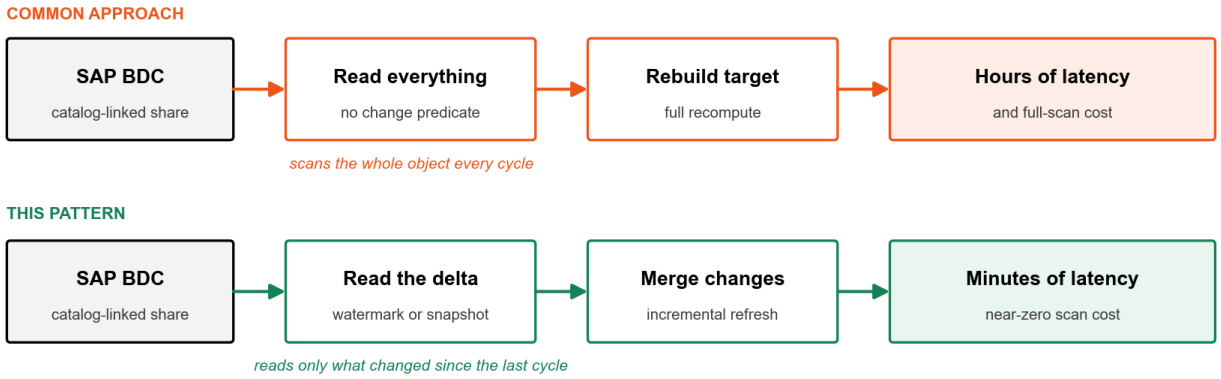
# Change Data Capture from SAP Business Data Cloud

Incremental capture from catalog-linked zero-copy shares in Snowflake - what works, what does not, and what it costs

**THE FINDING**

Change data capture from an SAP BDC catalog-linked share is achievable today, and the mechanism is an existing Snowflake platform feature rather than something to be built. Dynamic tables refresh incrementally over the share and establish that nothing has changed in a few hundred milliseconds without recomputing the source. A replication watermark is available for the narrow cases dynamic tables cannot serve. The three mechanisms most teams reach for first - streams, the CHANGES clause and change tracking - are all refused, for one reason that explains most of what follows: the shared objects are Iceberg tables on an external catalog.

**The problem: a zero-copy share carries no built-in change signal**




Most teams re-read the entire share on a schedule because nothing in a zero-copy share tells them what changed. The pattern in this summary reads only the delta.

## Why this matters commercially

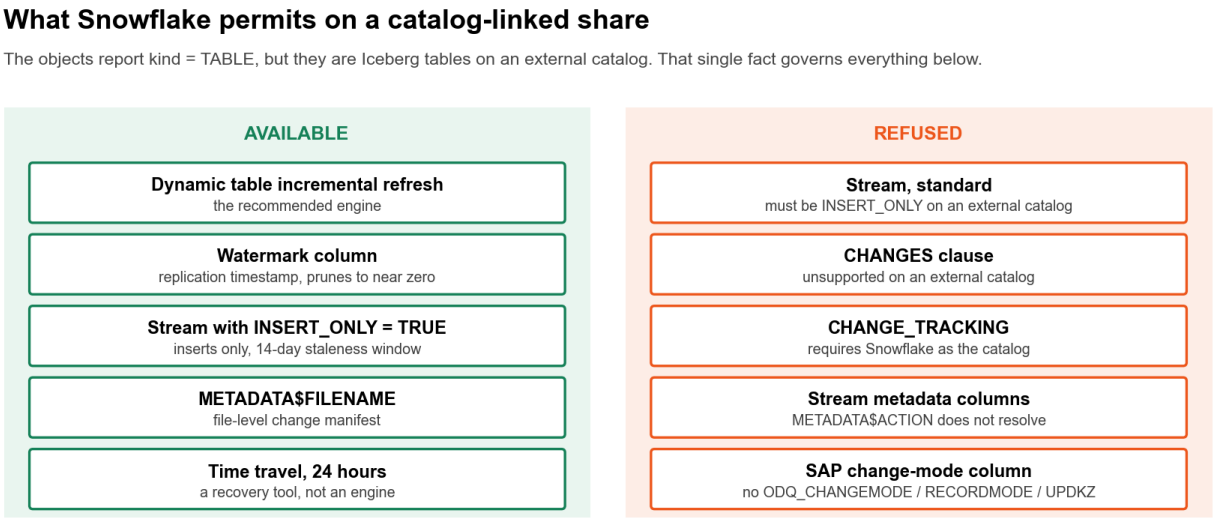
Two complaints usually arrive together from consumers of an SAP BDC share, and only one of them is a platform problem. **Latency** - the data is hours or a day behind - is solvable, and this is how. **Completeness** - a required measure or dimension is absent from the share - is not solvable at any refresh frequency, because the fields are missing rather than stale. Separating the two early avoids delivering fifteen-minute freshness on a dataset that cannot answer the question being asked.

## What a customer gets from adopting the pattern

- **Latency measured in minutes rather than hours.** A no-change refresh completes in a few hundred milliseconds, so a fifteen-minute target lag is affordable on transactional objects.
- **Near-zero incremental read cost.** On a reference tenant a watermark predicate returned the changed rows while scanning 0 MB of a 7 GB object. The intuitive alternative - filtering on a business date - scanned 4.7 GB for the same answer.
- **No engine to build or maintain.** The recommended mechanism is configuration of objects most customers already have in their Gold layer, plus one small control schema of three objects.
- **Correct handling of deletes and updates.** Dynamic tables diff source snapshots, so they capture rows the share has stopped carrying. A hand-built watermark engine cannot, because a BDC share carries no change-mode column.

## What the platform permits

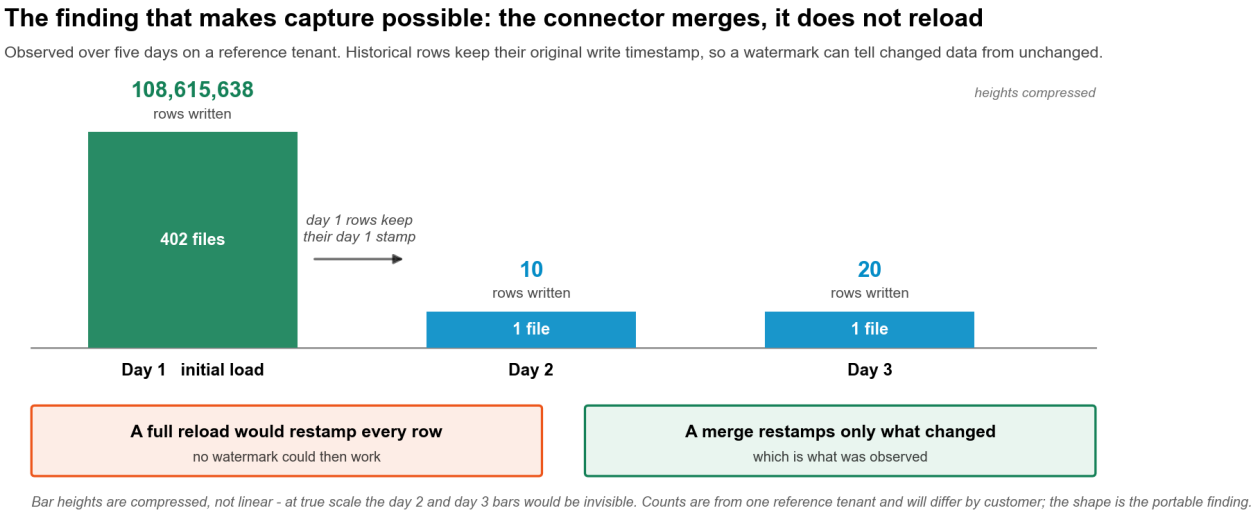
A catalog-linked database reports `kind = TABLE` and `is_external = N` on the objects inside it, which suggests ordinary tables and ordinary options. They are not ordinary tables. Snowflake's own refusals are the clearest statement of what is available.




Verified by attempting each feature. The refusal text is the evidence: for example, a standard stream is rejected with "Streams on External Tables or Iceberg tables with an external catalog must have INSERT_ONLY set to true".

## The precondition: the connector merges rather than reloads

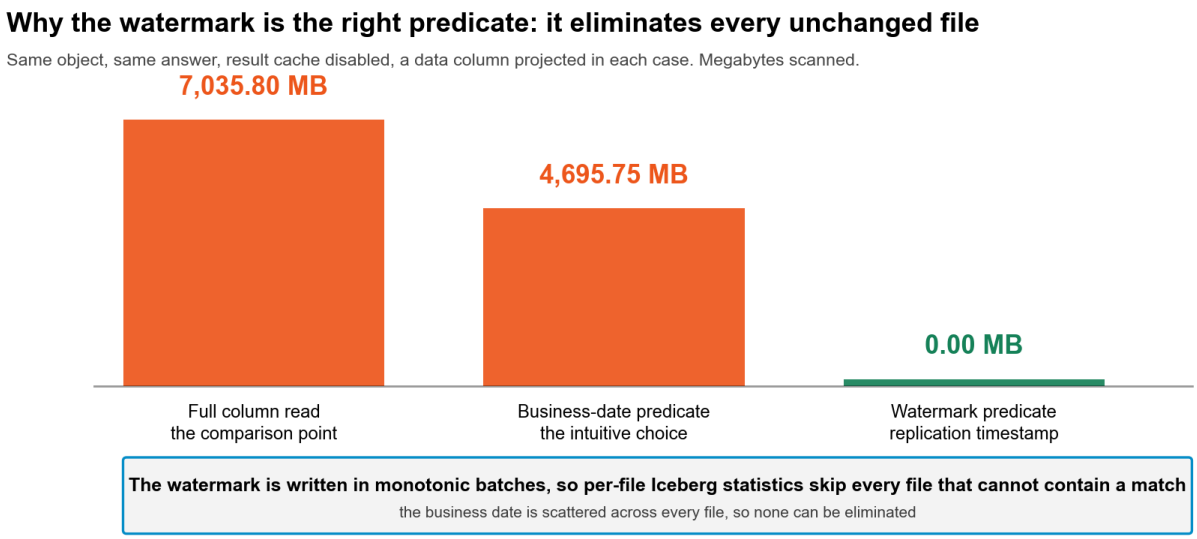
Everything rests on this. If the BDC connector rewrote the whole share on each cycle, every row would carry the latest write timestamp and nothing could separate changed data from unchanged. Observed over five days on a reference tenant, it does not: historical rows keep their original timestamp and new data lands in new files. This is a property of the pipeline rather than a platform guarantee, so it should be verified per customer - the assessment tool referenced at the end does exactly that.




Reference-tenant figures. Row counts will differ on any other customer; the shape - one bulk load, then small deltas in new files - is what generalises.

## Why the cost collapses

The connector writes the replication timestamp in monotonic batches, so each underlying Parquet file spans a narrow range of it. Per-file Iceberg statistics then eliminate every file that cannot contain a match. A business date has no such correlation with file layout - postings from every period are scattered across every file - so no file can be eliminated and the predicate degenerates into a full scan that discards almost everything it read.



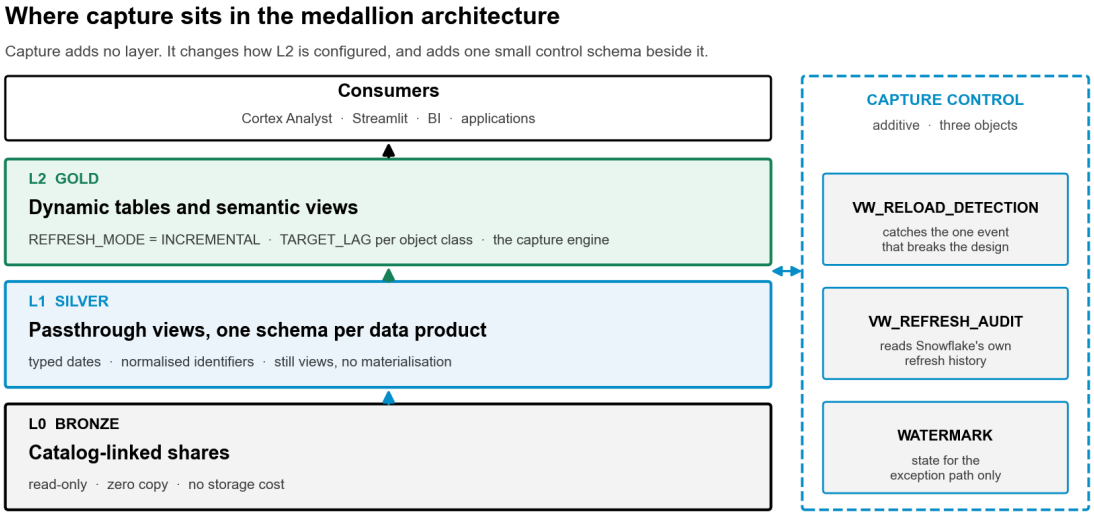

Reference tenant, result cache disabled, a data column projected in each case. COUNT(*) alone can be answered from Iceberg manifest statistics without reading data, which flatters the result and was deliberately avoided.

**THE ONE THING TO MONITOR**

The pruning is a property of how the connector currently writes, not a contractual guarantee. If the underlying files were ever compacted, rows with old timestamps would be rewritten into new files, per-file ranges would widen and pruning would degrade toward a full scan. The pattern therefore ships with a monitor whose only job is to detect that, and the connector switching to full reload - the two events that would silently make incremental capture as expensive as a rebuild.

## Where capture sits in the architecture

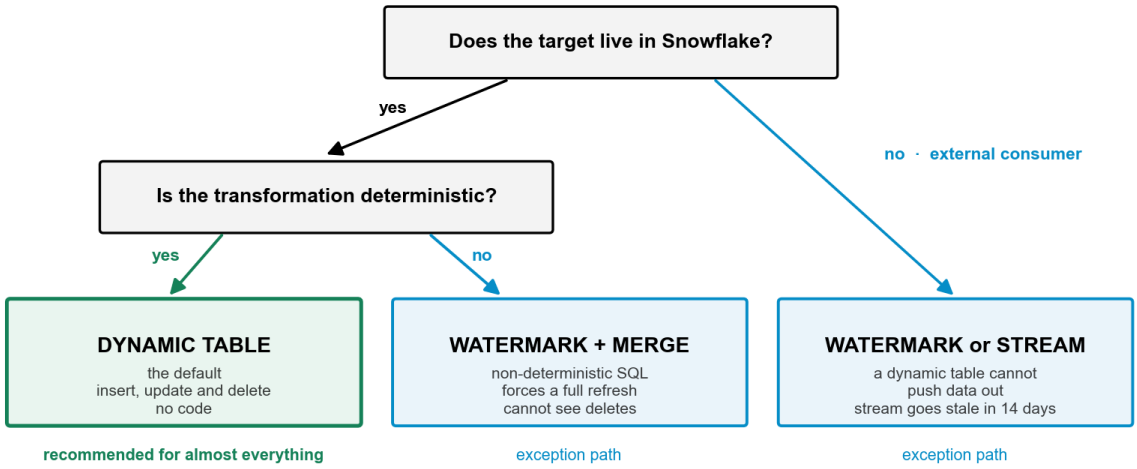
Capture adds no layer to a medallion architecture. The raw catalog-linked shares and the passthrough view layer above them are unchanged, and remain zero-copy with no storage cost. What changes is how the Gold layer is configured, plus one additive control schema. Incremental refresh was verified to survive the query shapes a real Gold layer uses: a flat projection over views that cast types, a multi-view join, an aggregate with GROUP BY, and a left join to a pre-aggregated child.




The two configuration choices that carry the weight: state REFRESH_MODE = INCREMENTAL explicitly rather than leaving it as AUTO, and set TARGET_LAG per object class rather than uniformly.

## Choosing a mechanism

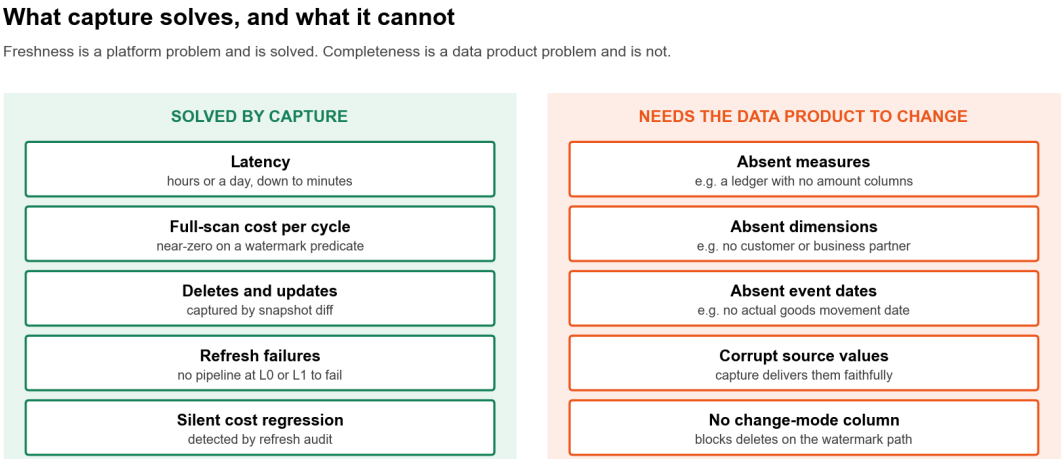
Three are available. One is the default; the other two exist for cases the default cannot serve.




Dynamic tables are the default for anything landing inside Snowflake. The watermark path exists for two specific cases, and is strictly weaker in one respect: it cannot detect physical deletes.

| | Dynamic table | Watermark + MERGE | INSERT_ONLY stream |
|---|---|---|---|
| Inserts and updates | yes | yes | yes |
| Deletes | **yes** - diffs snapshots | **no** - no change-mode column | **no** - by definition |
| Cheap no-change detection | yes | yes | yes |
| Code to write and maintain | none | task, control table, MERGE per object | task, MERGE per object |
| Can feed a target outside Snowflake | no | yes | yes |
| Goes stale if unread | no | no | yes, 14 days |
| Blocked by non-deterministic SQL | yes | no | no |

## What capture does not solve




Freshness is a platform problem and is solved. Completeness is a data product problem and is not - it requires the SAP-side data product definition to change.

## Recommended next steps

- **Assess, do not assume.** Run the readiness assessment against the customer's own account. It discovers the catalog-linked databases by kind rather than by name, so it is portable across tenants, and returns a verdict with the evidence behind it. Read-only by default.
- **Deploy the monitor first.** Before configuring anything, establish the baseline and the reload-detection view. It is the only thing that will tell you if the precondition ever stops holding.
- **State the refresh mode explicitly.** Leaving REFRESH_MODE as AUTO allows a silent fall back to a full refresh, with no error and nothing in the object's behaviour to point at. Stating INCREMENTAL turns a silent cost regression into a deployment-time error.
- **Separate the two conversations.** Latency is yours to fix and can be delivered against the share as it stands. Absent measures and dimensions are a data product change with a lead time outside your control, and should be raised with the SAP team in parallel rather than waited on.

---

Every measurement in this summary was taken against a live SAP BDC catalog-linked share on a Snowflake account over a five-day observation window, using an open assessment harness. Figures labelled as reference-tenant will differ on any other customer; the mechanisms and their limits are what generalise. Two questions remain open and are documented as such: what a refresh does when the source has genuinely changed, and whether an insert-only stream yields rows on a source change. Neither affects the recommendation. This is a community reference pattern, not a Snowflake product commitment. Prepared 19 August 2026.